SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2 **Duration: 45 Minutes**
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I ***Chandra Prakash D / 192524060*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWER SHEET

Artificial Intelligence (CSA17) – Assessment Tool 3: Descriptive Written Test

Name	Chandra Prakash D
Register No.	192524060
Course	Artificial Intelligence (CSA17) – SIMATS Engineering
Course Outcome	CO1 – Intelligent agents and uninformed search methods (BL3)

Question 1: Intelligent Agents in a Smart Home System

1.1 Introduction

An **intelligent agent** perceives its environment through sensors and acts on it through actuators to maximise its performance measure. The agent types differ in how much they remember, whether they hold goals, and how they judge between choices. For the smart home, the task description (PEAS) is:

Performance measure	User comfort, low energy consumption and cost, security of the home and its occupants, and adaptation to user habits.
Environment	Rooms, occupants, weather, time of day, electricity tariff – partially observable, dynamic, stochastic (people behave unpredictably), continuous and sequential.
Sensors (percepts)	Motion/occupancy, temperature, humidity, light level, clock, door and window contacts, camera, smoke/gas, user app input and phone location.
Actuators (actions)	Switch/dim lights, set thermostat/AC/heater, open/close blinds, lock doors, arm the alarm, sound the siren, send notifications.

1.2 Types of intelligent agents and how each works in the smart home

(a) Simple reflex agent

It selects an action only from the *current percept* using condition–action rules (if–then), and keeps no memory. **Smart-home examples:** if motion is detected and the room is dark → switch the light on; if smoke is detected → sound the alarm; if a door opens while the system is armed → trigger the siren; if temperature > 30 °C → start the fan.

- **Strengths:** very fast, simple, cheap and predictable – ideal for emergencies.
- **Weaknesses:** works only when the correct action can be decided from the current percept. A motionless person reading in a room makes the motion sensor go quiet, so the lights switch off; the thermostat can switch on and off repeatedly; it cannot learn or distinguish ‘nobody home’ from ‘nobody moving’.

(b) Model-based reflex agent

It keeps an **internal state** (a model of how the world evolves and how its actions affect it) so that it can handle partial observability, and then applies rules to that state. **Smart-home examples:** it remembers the time of the last motion in each room and keeps the lights on for 10 more minutes; infers that the house is empty if no motion has been seen for 30 minutes and the phone is away; tracks the temperature trend and window state (does not heat while a window is open).

- **Strengths:** removes most of the flaws of the simple reflex agent; copes with missing or noisy sensor data.
- **Weaknesses:** still follows fixed rules – it has no explicit objectives and cannot plan ahead or weigh conflicting aims.

(c) Goal-based agent

It has an explicit **goal** describing a desirable state and uses search and planning to choose actions that reach it. **Smart-home examples:** goal 'living room at 24 °C when the user arrives at 6 pm' → the agent works out when to start the AC (using the room's heating rate) and the order of actions; goal 'house secure for the night' → close blinds, lock all doors, arm sensors, turn off lights (a planned sequence); an 'away mode' goal.

- **Strengths:** flexible – the behaviour changes simply by changing the goal; can look ahead.
- **Weaknesses:** a goal is only 'achieved' or 'not achieved'. It cannot say which of several ways is better (cheaper, quieter, more comfortable) or handle goals that conflict, such as comfort versus energy saving.

(d) Utility-based agent

It uses a **utility function** that maps each possible state to a real number measuring 'how happy' the user would be, and chooses the action with the highest *expected utility*, which also handles uncertainty. A suitable function is $U = w_1 \cdot Comfort - w_2 \cdot Energy\ cost - w_3 \cdot Security\ risk$, where the weights express the user's priorities. **Smart-home examples:** pre-cool the house during the cheaper off-peak tariff and accept a 1 °C deviation to save energy; dim the lights slightly when daylight is available; choose between heating the whole house or only occupied rooms based on the probability that the user returns.

- **Strengths:** makes rational trade-offs between comfort, cost and safety; works under uncertainty; the weights can be learned from the user.
- **Weaknesses:** needs a good utility function and probability estimates; more computation; choices can be harder to explain.

1.3 Comparison for the smart home

Agent type	Comfort	Energy efficiency	Security	Suitable role
Simple reflex	Low – no context	Low – wastes energy	High for instant alarms	Emergency triggers
Model-based	Medium	Medium	Medium – knows home/away	State tracking (occupancy)
Goal-based	High at scheduled times	Medium	High – planned routines	Routines and scheduling
Utility-based	High	High – cost-aware	High – risk-weighted	Trade-off decisions

1.4 Most effective approach: a hybrid layered agent

No single type satisfies all three requirements, so the best design is a **hybrid agent built in layers**, with a **utility-based decision maker on top of a model-based core**, a goal-based planner for routines, a learning element, and a simple reflex safety layer that has priority.

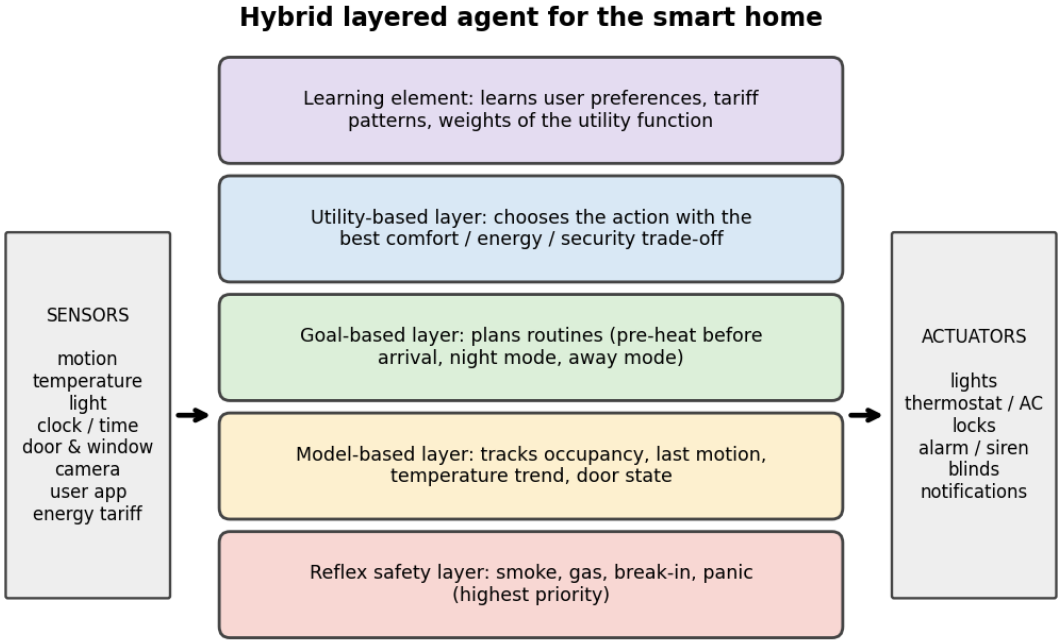


Figure: *Hybrid layered agent. The lowest layer has priority; higher layers refine comfort and energy decisions.*

Requirement	How the hybrid agent satisfies it
Security	The reflex layer reacts instantly to smoke, gas or intrusion without waiting for planning and overrides all other layers. The model tracks home/away state to arm the system automatically, and the goal layer runs night/away routines.
Comfort	The model estimates occupancy and room temperature trends; the goal layer pre-conditions rooms before the user arrives; the utility layer balances temperature and lighting against user preferences.
Energy efficiency	The utility function penalises energy cost, so the agent uses off-peak tariffs, switches devices off in empty rooms and accepts small comfort deviations when the saving is large.
Learning user preferences	The learning element observes manual overrides (for example the user lowers the thermostat every night) and updates the utility weights and the model, so behaviour improves over time.

Question 2: Robot in an Unknown Maze – UCS, IDS and Agent Design

2.1 The problem

The maze can be viewed as a graph in which each junction or cell is a state, each possible move is an action, and the exit is the goal. Because the layout is unknown, the robot is an **online, exploring agent**: it discovers the graph while moving, sees only nearby walls (partially observable), and wrong moves cost real travel time and energy. **Uninformed search** is appropriate because nothing tells the robot which direction is closer to the exit, so no heuristic is available. The robot builds a map of what it has seen and runs the search on that map.

2.2 Uniform Cost Search (UCS)

UCS always expands the frontier node with the lowest path cost g(n), using a priority queue. In a maze with equal step costs it behaves like Breadth-First Search; when moves have different costs (turns, slopes, narrow corridors, wet floor, risky areas) it finds the **least-cost route** rather than the one with the fewest steps.

- **Complete:** yes, if every step cost $\geq \varepsilon > 0$ and the branching factor is finite.
- **Optimal:** yes – nodes are expanded in order of path cost, so the first time the exit is removed from the queue the path is cheapest.
- **Time:** $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$, where b = branching factor, C^* = cost of the optimal solution, ε = smallest step cost. It can be much larger than b^d when there are many cheap steps.
- **Space:** also $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$ – it must store the entire frontier and the explored set.

Advantages: guarantees the cheapest path; handles variable costs; complete. **Disadvantages:** exponential memory (a serious limit for a small robot); explores in all directions equally; slow when step costs are tiny; needs a visited set to avoid cycles in mazes with loops.

2.3 Iterative Deepening Search (IDS)

IDS runs depth-limited depth-first search with limits 0, 1, 2, 3, … until the exit is found. It combines the small memory of depth-first search with the completeness and shallowest-solution property of breadth-first search.

- **Complete:** yes, when the branching factor is finite (avoids DFS getting lost in infinite or very deep branches).
- **Optimal:** yes when all step costs are equal (it finds the shallowest exit); not for varying costs.
- **Time:** $O(b^d)$, d = depth of the shallowest exit. The repeated work is small: with b = 10 and d = 5 IDS generates 123,456 nodes compared with 111,111 for BFS (only about 11 % more), because most nodes lie at the deepest level.
- **Space:** $O(b \cdot d)$ – linear, it stores only the current path and the siblings along it.

Advantages: very small memory; complete; finds the shallowest exit; simple to implement. **Disadvantages:** re-generates upper levels each iteration; ignores step costs; with cycles in the maze it must check for repeated states on the current path (or store a visited set, which gives up part of the memory saving).

Pseudocode

```
function ITERATIVE-DEEPENING-SEARCH(problem)
  for depth = 0 to infinity do
     result <- DEPTH-LIMITED-SEARCH(problem, depth)
     if result != cutoff then return result      // solution or failure

function UNIFORM-COST-SEARCH(problem)
  frontier <- priority queue ordered by g, containing start
  explored <- empty set
  loop: if frontier is empty then return failure
        node <- POP(frontier)
        if GOAL-TEST(node) then return SOLUTION(node)
        add node to explored
        for each child: if new (or cheaper) then INSERT(frontier, child)
```

2.4 Comparison of UCS and IDS

Property	UCS	IDS
Data structure	Priority queue (frontier + explored set)	Stack / recursion (current path only)
Completeness	Yes (step cost ≥ ε > 0)	Yes (finite branching)
Optimality	Yes – least total cost	Only for equal step costs (shallowest)
Time complexity	O(b^(1+⌊C*/ε⌋))	O(b^d)
Space complexity	O(b^(1+⌊C*/ε⌋)) – exponential	O(b·d) – linear
Handles varying costs	Yes	No
Best used when	Costs differ and memory is available	Memory is limited and costs are uniform

2.5 Relating the maze to the types of intelligent agents

Agent type	Behaviour in the maze	Effect on decisions
Simple reflex	Rules such as 'if wall on left, turn left' (wall-following) based only on the current percept.	Fast but has no memory; can loop forever in mazes with cycles or islands and cannot guarantee a shortest path.
Model-based	Builds a map of visited cells, marks dead ends and remembers junctions.	Avoids repeating cells and handles partial observability; makes systematic search possible.
Goal-based	Goal = reach the exit; plans a route on the known map using UCS or IDS.	Decisions are purposeful; can re-plan when a path is blocked.
Utility-based	Scores routes by time, energy, turning cost and risk, then picks the best trade-off.	Chooses the best path, not just any path; fits the UCS cost idea.
Learning	Improves on repeated runs by remembering the maze and good strategies.	Later runs are faster because exploration is reduced.

How agent design affects decision-making: the agent's *memory* decides whether it can recognise repeated states; its *goal* lets it plan rather than just react; its *utility function* lets it compare routes by cost; and its *sensing* (what percepts are available) decides whether it must explore physically or can search on a known map. A poorly designed agent (simple reflex) wastes effort or never finishes, whereas a model-based, goal-based agent with a cost model makes efficient and reliable decisions.

2.6 Recommended combination and justification

Recommended: a model-based, goal-based agent with a utility (cost) function, using IDS for exploration while the maze is unknown (uniform costs) and UCS to compute the least-cost route on the map it has learned.

- **Model-based:** the maze is unknown and only partly visible, so the robot must keep a map to avoid loops and dead ends.
- **Goal-based:** reaching the exit is an explicit goal that needs planning, not just reflexes.
- **Utility/cost awareness:** when moves have different costs (turns, slopes, energy), UCS gives the cheapest path, and it can be re-run as the map grows.
- **IDS during exploration:** its linear memory $O(b \cdot d)$ suits the robot's small onboard memory, and its shallowest-first behaviour finds the nearest exit without being trapped in deep dead ends.
- **Efficient in practice:** IDS iterations are performed on the stored map (in memory), not by physically re-walking the maze; the robot physically travels only along the chosen path. Once the exit has been found, UCS on the learned map gives the optimal path for later runs.

If memory were plentiful and costs varied a lot, UCS alone would be preferred; if costs were uniform and memory very limited, IDS alone would be enough. The combination covers both situations and is therefore the most robust choice.